

Multi-Rail Isolated Power Supply Monitoring System

Michael Capotosto

August 7, 2026

Rev. 1

Contents

1	Introduction	1
2	Installing EPICS Base and IOCs	3
2.1	Introduction	3
2.2	EPICS Installation Overview	3
2.3	Installing EPICS Base and Required Modules	3
2.4	Installing the PSU Monitor IOC	5
3	CS-Studio - Phoebus	7
3.1	Introduction	7
3.2	Installing Phoebus	7
4	Prototyping and Development	9
4.1	Introduction	9
4.2	Testing the LTC2945	9
4.2.1	Calculating Shunt Value for Test	10
4.2.2	Pictures of Evaluation Board, Display, and Phoebus Panels	11

Chapter 1

Introduction

Chapter 2

Installing EPICS Base and IOCs

2.1 Introduction

A simple IOC was written to interface the Arduino Giga which monitors the six supplies. The following section details how to install EPICS Base and required modules, and compile the IOC. It is written assuming a Linux environment; this particular build was done on Ubuntu 22.04LTS.

2.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- EPICS_BASE=/epics/base or /epics/base_[version] or similar
- SUPPORT=/epics/support – or sometimes MODULES instead of SUPPORT is used.

Modules required for the PSU monitor are:

- pscdrv: Portable Streaming Controller Driver
- epics/iocs - each IOC is cloned into this repository
- epics/systemd-softioc - This is a utility for managing IOCs as a system service

2.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:

```
sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev
autoconf automake libtool pkg-config
```
2. Create the EPICS user group, and the empty EPICS directory:

```
sudo groupadd epics
sudo usermod -aG epics $USER
```

```
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into /epics/base via:

```
git clone --recursive -b 7.0
  https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:

```
make clean
make
```

5. Create the remaining directory structure:

```
mkdir -p /epics/iocs /epics/modules
```

6. Clone PSC Driver:

- (a) `cd /epics/modules`
- (b) `git clone https://github.com/mdaidsaver/pscdrv.git pscdrv`
- (c) `cd /epics/modules/pscdrv`
- (d) Set up required files in `configure/RELEASE`, `configure/RELEASE.LOCAL`
- (e) Build with `make clean`, and `make`

7. Clone Manage IOCs Utility:

- (a) `cd /epics`
- (b) `git clone https://github.com/NSLS2/systemd-softioc.git`
- (c) Navigate to the repo directory `cd systemd-softioc`
- (d) Make the install script executable and run:


```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

2.4 Installing the PSU Monitor IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
`git clone https://github.com/capotosto/PS_Monitor psuMonitorIOC`
3. `cd psuMonitorIOC`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. `echo -e "EPICS_BASE = /epics/base\nPSCDRV = /epics/modules/pscdrv\n"`

6. Build the IOC: `make clean, make`
7. Update the `st.cmd` file as needed: `cd iocBoot/iocpsuMonitor`
8. Create the `st.cmd` file used by `systemd-softioc`:

```
echo -e '#!/bin/bash\n\n
cd "/epics/iocs/psuMonitorIOC/iocBoot/iocpsuMonitor"\n./st.cmd' > st.cmd
```

9. Make the new `st.cmd` executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the config file: `nano config`
12. Set the `NAME=`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level `st.cmd` to the `iocBoot/iocpsuMonitor/st.cmd`
14. Enroll in `manage-iocs` as a service: `sudo manage-iocs install psuMonitorIOC`
15. Enable auto-start on boot: `sudo manage-iocs enable psuMonitorIOC`
16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`

Chapter 3

CS-Studio - Phoebus

3.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directories, `/epics/iocs/PSCtest/gui` and `/epics/iocs/zpsc_ioc/gui`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'PSC Launcher' panel using the Home button, instead of having to manually navigate to the file when needed.

3.2 Installing Phoebus

1. If not installed, install Java, e.g.:

```
sudo apt update && sudo apt install openjdk-17-jdk
```
2. Create the directory: `sudo mkdir /opt/phoebus`

```
sudo chown $USER:epics /opt/phoebus
```

```
chmod 2775 /opt/phoebus
```
3. Download Phoebus and extract to the `/opt/phoebus` directory

```
https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz
```

```
cd /opt/phoebus
```

```
wget https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz
```

```
tar -xvzf phoebus-4.7.4-linux.tar.gz -C . --strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`

5. Alias Phoebus to run via command `run-phoebus`:

```
echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' >> ~/.bashrc  
source ~/.bashrc
```

6. Now test! Try `run-phoebus` from a terminal.

Chapter 4

Prototyping and Development

4.1 Introduction

4.2 Testing the LTC2945

The LTC2945 testing was carried out using a Linear Tech eval board, DEMO CIRCUIT 1697A. Connections and modifications were made as follows:

- VIN: tied to Arduino +5V
- GND: tied to Arduino GND
- ADIN: Tied to GND
- SCL: Tied to Arduino Pin D8 I2C2 SCL
- SDAI/SDAO (JP2 set to TIE): Arduino Pin D9 I2C2 SDA
- VPU: Tied to Arduino +3v3
- VOUT: Float or tied to dummy load ($1\text{k}\Omega$ to $10\text{k}\Omega$)
- $20\text{m}\Omega$ shunt replaced with 17Ω shunt for testing with $\sim 5\text{k}\Omega$ load
- Installed $4.7\text{k}\Omega$ pull up on I2C lines via R4, R5 footprints on EVB

4.2.1 Calculating Shunt Value for Test

Using the 5V from the arduino as a source, and a load ranging 1k to 10k ohms, we calculate:

$$R_{L,min} = 1k\Omega$$

$$V_{shunt} = V_S \frac{R_S}{R_L + R_S}$$

$$R_S = \frac{V_{Shunt} R_L}{V_s - V_{Shunt}}$$

$$V_{shunt} = 0.1024V$$

$$R_{S,fullscale} = \frac{(0.1024V)(1k\Omega)}{5V - 0.1024V} = 20.91\Omega$$

Adding some headroom, say, 15%, and going to an E96 resistor value...

$$R_S = 0.85(20.91) = 17.8\Omega$$

Verifying for the high end of the range:

$$I_{min} = \frac{5V}{10k\Omega + 17.8\Omega} = 499.1\mu A$$

$$V_{shunt,min} = (499.1\mu A)(17.8\Omega) = 8.88mV$$

The shunt will range from 8.88mV to 87.44mV, within the ADC's usable range of 0V to 102.4mV.

4.2.2 Pictures of Evaluation Board, Display, and Phoebus Panels

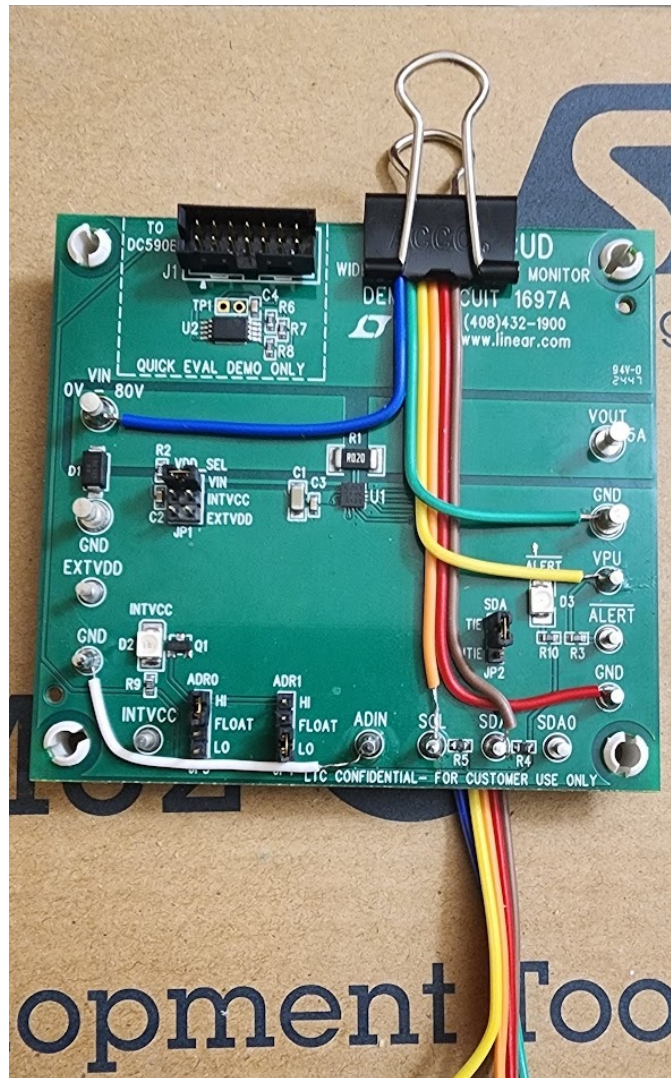


Figure 4.1: DEMO CIRCUIT 1697A Eval Board with Original Shunt, fly leads to connect to Arduino.



Figure 4.2: DEMO CIRCUIT 1697A Eval Board with 17Ω Shunt, fly leads to connect to Arduino, and $4.7k\Omega$ test load.

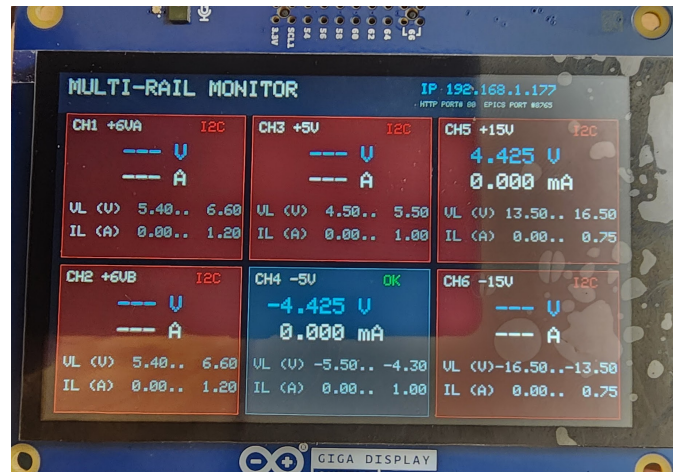


Figure 4.3: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm

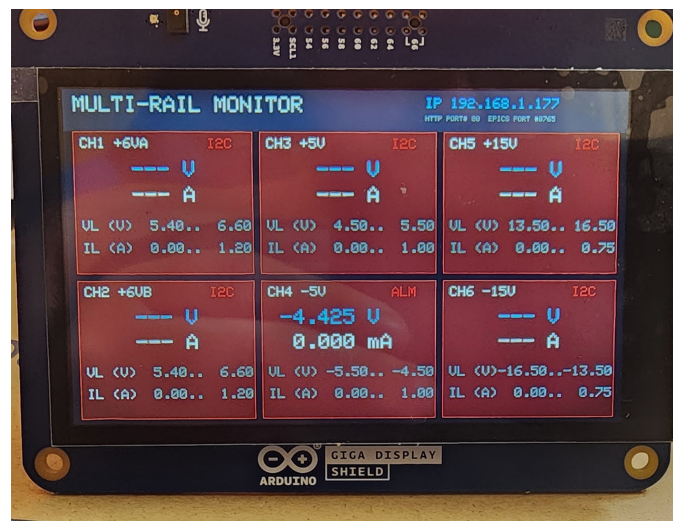


Figure 4.4: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, with UVL alarm, other channels I2C ALARM.

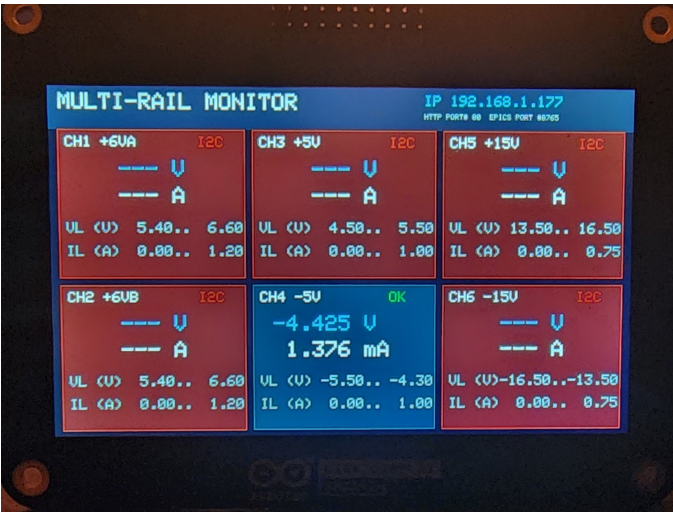


Figure 4.5: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 4.6: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load